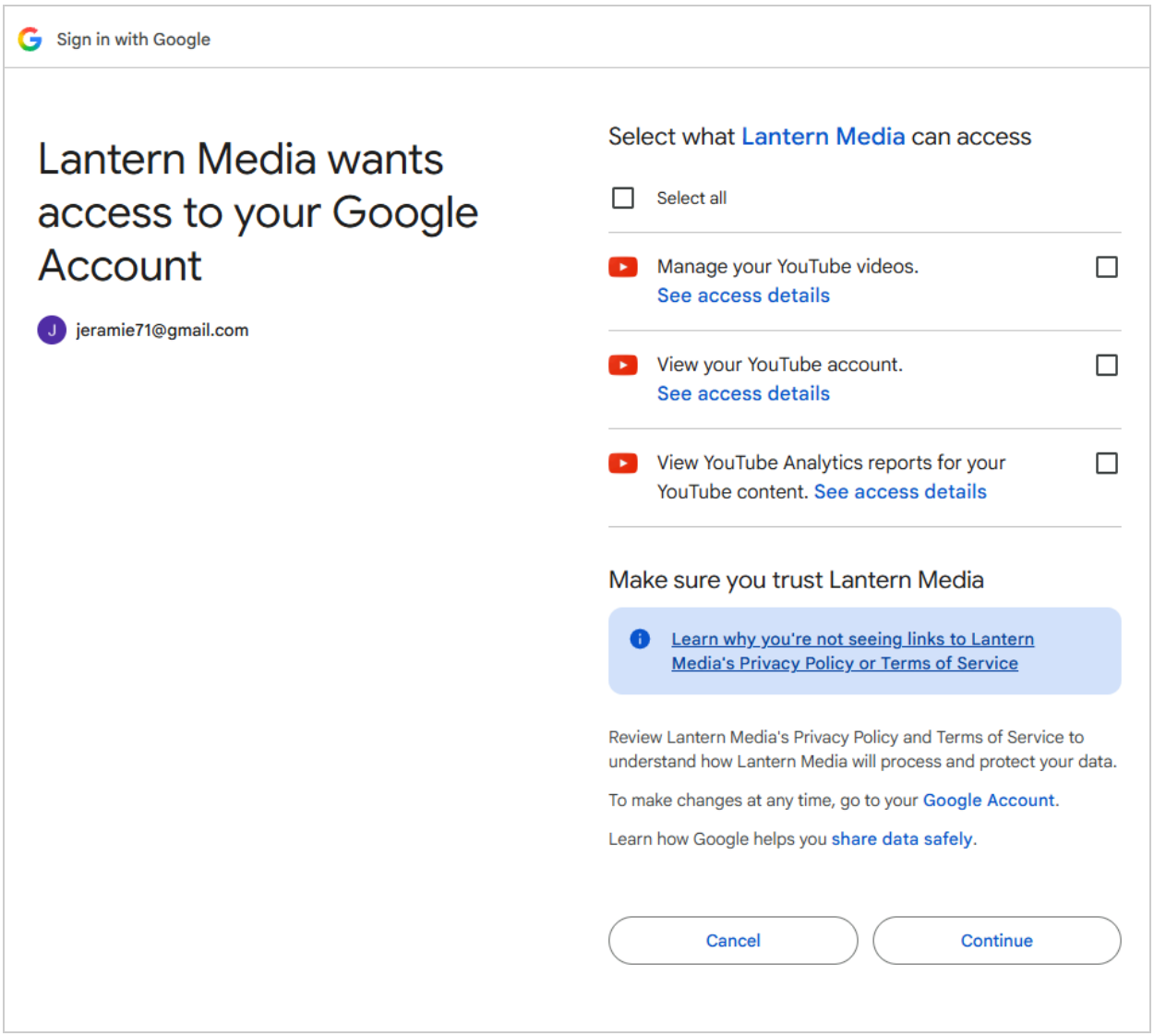
Lantern Publisher - Conditional Evidence for Project #1 (534783899062)

Applicant: 10X Lead Pro LLC · API client: Lantern Publisher · Google Cloud project: lantern-media-513512 · Primary website: https://10xleadpro.com · Privacy policy: https://10xleadpro.com/privacy.html

Lantern Publisher is an internal-only command-line tool with no user interface, no login page and no third-party users. It uploads the applicant's own videos to the applicant's own YouTube channels and reads back the applicant's own analytics. This document contains: (a) OAuth flow evidence, (b) the complete working upload script, and (c) architecture and user-flow diagrams.

(a) OAuth flow - consent screen and scopes

Consent screen shown to the operator (the only user) when authorizing Lantern Publisher. Scopes requested: `youtube.upload`, `youtube.readonly`, `yt-analytics.readonly`.



Revocation

Access is revoked by the operator at any time from the Google Account → Security → Third-party apps & services page (https://myaccount.google.com/connections). Lantern stores the refresh token only in Windows Credential Manager (`LANTERN_YT_REFRESH_TOKEN_*`); revocation causes the next API call to fail with `invalid_grant`, the publisher marks the channel as unauthorized and stops, and stored data for that account is deleted within 30 days (see privacy policy, "YouTube API Services").

(b) Complete working upload script - services/publisher/upload_video.py

This is the full file as deployed (not a snippet). Behaviour: resumable `videos.insert`; `privacyStatus` defaults to *private*; `selfDeclaredMadeForKids` set per channel; `containsSyntheticMedia` set true; every upload is backed by a durable `operations` row so a retry reconciles against YouTube instead of re-posting; a `--confirm-upload` guard prevents accidental uploads.

```
#!/usr/bin/env python3
"""
Resumable YouTube Data API v3 upload, built for the week-2A audit package
(docs/audit/form_answers.md references this file as the upload
implementation the compliance audit describes).

Auth: a Desktop OAuth client (C:\\Lantern\\secrets\\youtube_oauth_client.json,
never read here directly -- see _read_oauth_client) plus a stored refresh
token. scripts/youtube_auth.py is what produces that refresh token, stored
in Windows Credential Manager as LANTERN_YT_REFRESH_TOKEN_A; this script
reads it the same way every other container secret reaches Lantern code
(docs/secrets.md): a file at /run/secrets/<name>, falling back to an env
var for standalone testing outside Docker.

Safety gate, per CLAUDE.md ("Never publish anything to YouTube... Week 1
is engine only" -- still the rule in week 2A): without --confirm-upload
this script makes NO network call at all. It validates every argument,
builds the exact video resource body it would send, derives the
idempotency key, and upserts the operations row as 'pending' with that
body recorded -- then stops. Only --confirm-upload proceeds to refresh
the OAuth token, open a resumable session and PUT video bytes. Never pass
--confirm-upload this week.

Usage (dry run -- the only mode used this week):
    python upload_video.py --video path/to/video.mp4 \\
        --title "..." --description "..." --channel-id <channels.id uuid> \\
        --episode-id <episodes.id uuid> --privacy-status private \\
        --made-for-kids false --contains-synthetic-media true \\
        --tags "homestead,offgrid"

Real upload (never run this week):
    python upload_video.py ... --confirm-upload
"""
import argparse
import hashlib
import json
import mimetypes
import os
import sys
import time
import urllib.error
import urllib.request
import uuid

import psycopg2
import psycopg2.extras

OAUTH_CLIENT_PATH = os.environ.get(
    "YOUTUBE_OAUTH_CLIENT_PATH", r"C:\Lantern\secrets\youtube_oauth_client.json"
)
TOKEN_URL = "https://oauth2.googleapis.com/token"
UPLOAD_URL = "https://www.googleapis.com/upload/youtube/v3/videos?uploadType=resumable&part=snippet,status"
CHUNK_SIZE = 8 * 1024 * 1024  # 8 MiB, a multiple of 256 KiB as the API requires


class UploadError(Exception):
    pass


# ---- secrets ---------------------------------------------------------------

def _read_secret(name: str, env_fallback: str) -> str:
    """Same lookup order as every other Lantern service (dashboard/app/auth.py,
    services/llm-router/llm_router/router.py): a file under /run/secrets first
    (how Docker secrets and launch.ps1's tmpfs secrets arrive), an env var
    second (for running this script standalone outside Docker)."""
    path = f"/run/secrets/{name}"
    try:
        with open(path, "r", encoding="utf-8") as f:
            value = f.read().strip()
            if value:
                return value
    except FileNotFoundError:
        pass
    return os.environ.get(env_fallback, "")
```

```
def _read_oauth_client() -> tuple[str, str]:
    try:
        with open(OAUTH_CLIENT_PATH, "r", encoding="utf-8") as f:
            data = json.load(f)
    except FileNotFoundError as exc:
        raise UploadError(f"OAuth client JSON not found at {OAUTH_CLIENT_PATH}") from exc
    # Desktop client JSON nests id/secret under "installed".
    section = data.get("installed") or data.get("web") or {}
    client_id = section.get("client_id", "")
    client_secret = section.get("client_secret", "")
    if not client_id or not client_secret:
        raise UploadError("OAuth client JSON is missing client_id/client_secret")
    return client_id, client_secret


def _refresh_access_token(client_id: str, client_secret: str, refresh_token: str) -> str:
    import urllib.parse

    data = urllib.parse.urlencode({
        "client_id": client_id,
        "client_secret": client_secret,
        "refresh_token": refresh_token,
        "grant_type": "refresh_token",
    }).encode("utf-8")
    req = urllib.request.Request(TOKEN_URL, data=data, method="POST")
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            payload = json.loads(resp.read())
    except urllib.error.HTTPError as exc:
        raise UploadError(f"token refresh failed: {exc.code} {exc.read().decode('utf-8', 'replace')}") from exc
    access_token = payload.get("access_token")
    if not access_token:
        raise UploadError(f"token refresh response had no access_token: {payload}")
    return access_token


# ---- operations table (durable record of this external action) ------------

def _db():
    return psycopg2.connect(os.environ["DATABASE_URL"])


def _idempotency_key(channel_id: str, episode_id: str | None, video_path: str) -> str:
    # channel + episode + step, per technical_plan.md's idempotency-key rule;
    # falls back to a content hash when there's no episode_id (manual test upload).
    if episode_id:
        return f"upload-{channel_id}-{episode_id}"
    digest = hashlib.sha256(video_path.encode("utf-8")).hexdigest()[:16]
    return f"upload-{channel_id}-adhoc-{digest}"


def _get_operation(conn, idempotency_key: str) -> dict | None:
    with conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor) as cur:
        cur.execute("SELECT * FROM operations WHERE idempotency_key = %s", (idempotency_key,))
        return cur.fetchone()


def _upsert_operation(conn, idempotency_key: str, channel_id: str, episode_id: str | None,
                      state: str, request: dict, response: dict | None = None,
                      external_id: str | None = None):
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO operations (idempotency_key, kind, provider, external_id, channel_id,
                                    episode_id, state, request, response)
            VALUES (%s, 'youtube_upload', 'youtube', %s, %s, %s, %s, %s, %s)
            ON CONFLICT (idempotency_key) DO UPDATE SET
                state = EXCLUDED.state,
                external_id = COALESCE(EXCLUDED.external_id, operations.external_id),
                request = EXCLUDED.request,
                response = COALESCE(EXCLUDED.response, operations.response),
                updated_at = now()
            """,
            (idempotency_key, external_id, channel_id, episode_id, state,
             psycopg2.extras.Json(request), psycopg2.extras.Json(response) if response else None),
        )
    conn.commit()


# ---- video resource body --------------------------------------------------

def _build_video_body(args) -> dict:
    return {
        "snippet": {
            "title": args.title,
            "description": args.description,
            "tags": [t.strip() for t in args.tags.split(",") if t.strip()] if args.tags else [],
```

```
            "categoryId": args.category_id,
        },
        "status": {
            "privacyStatus": args.privacy_status,
            "selfDeclaredMadeForKids": args.made_for_kids,
            # YouTube's altered/synthetic-content disclosure flag -- required
            # whenever the video contains AI-generated or AI-altered media,
            # which is every Lantern upload.
            "containsSyntheticMedia": args.contains_synthetic_media,
        },
    }


# ---- resumable upload protocol ----------------------------------------------

def _create_resumable_session(access_token: str, body: dict, file_size: int, mime_type: str) -> str:
    data = json.dumps(body).encode("utf-8")
    req = urllib.request.Request(
        UPLOAD_URL, data=data, method="POST",
        headers={
            "Authorization": f"Bearer {access_token}",
            "Content-Type": "application/json; charset=UTF-8",
            "X-Upload-Content-Type": mime_type,
            "X-Upload-Content-Length": str(file_size),
        },
    )
    try:
        with urllib.request.urlopen(req, timeout=30) as resp:
            session_uri = resp.headers.get("Location")
    except urllib.error.HTTPError as exc:
        raise UploadError(f"resumable session create failed: {exc.code} {exc.read().decode('utf-8', 'replace')}") from exc
    if not session_uri:
        raise UploadError("resumable session create succeeded but returned no Location header")
    return session_uri


def _query_resumable_offset(session_uri: str, file_size: int) -> int:
    """Asks the session where it left off (used to resume after a dropped
    connection instead of re-uploading from byte 0 -- the resumable-upload
    reconciliation technical_plan.md asks for)."""
    req = urllib.request.Request(
        session_uri, data=b"", method="PUT",
        headers={"Content-Range": f"bytes */{file_size}"},
    )
    try:
        urllib.request.urlopen(req, timeout=30)
        return file_size  # 200/201 here means it's already fully uploaded
    except urllib.error.HTTPError as exc:
        if exc.code == 308:
            range_header = exc.headers.get("Range")
            if range_header:
                return int(range_header.split("-")[1]) + 1
            return 0
        raise UploadError(f"resumable offset query failed: {exc.code} {exc.read().decode('utf-8', 'replace')}") from exc


def _upload_chunks(session_uri: str, video_path: str, file_size: int, mime_type: str, start_offset: int) -> dict:
    offset = start_offset
    with open(video_path, "rb") as f:
        f.seek(offset)
        while offset < file_size:
            chunk = f.read(CHUNK_SIZE)
            chunk_len = len(chunk)
            end = offset + chunk_len - 1
            req = urllib.request.Request(
                session_uri, data=chunk, method="PUT",
                headers={
                    "Content-Length": str(chunk_len),
                    "Content-Range": f"bytes {offset}-{end}/{file_size}",
                    "Content-Type": mime_type,
                },
            )
            try:
                with urllib.request.urlopen(req, timeout=120) as resp:
                    # Final chunk: 200/201 with the created video resource.
                    return json.loads(resp.read())
            except urllib.error.HTTPError as exc:
                if exc.code == 308:
                    range_header = exc.headers.get("Range")
                    offset = int(range_header.split("-")[1]) + 1 if range_header else offset + chunk_len
                    continue
                raise UploadError(f"chunk upload failed at offset {offset}: {exc.code} {exc.read().decode('utf-8',
'replace')}") from exc
    raise UploadError("upload loop exited without a final response (unexpected)")


# ---- main -------------------------------------------------------------------

def parse_args(argv=None):
```

```
    p = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
    p.add_argument("--video", required=True, help="path to the rendered video file")
    p.add_argument("--title", required=True)
    p.add_argument("--description", default="")
    p.add_argument("--tags", default="", help="comma-separated")
    p.add_argument("--category-id", default="22", help="YouTube video category id (default 22 = People & Blogs)")
    p.add_argument("--channel-id", required=True, help="Lantern channels.id (uuid), not the YouTube channel id")
    p.add_argument("--episode-id", default=None, help="Lantern episodes.id (uuid), if this upload is for a tracked episode")
    p.add_argument("--privacy-status", default="private", choices=["private", "unlisted", "public"])
    p.add_argument("--made-for-kids", type=lambda s: s.lower() == "true", default=False)
    p.add_argument("--contains-synthetic-media", type=lambda s: s.lower() == "true", default=True)
    p.add_argument("--refresh-token-secret", default="youtube_refresh_token_a",
                   help="secret name to read the OAuth refresh token from (matches /run/secrets/<name> and docs/secrets.md's
LANTERN_ naming)")
    p.add_argument("--confirm-upload", action="store_true",
                   help="actually call the YouTube API. Never pass this this week.")
    return p.parse_args(argv)


def main(argv=None):
    args = parse_args(argv)

    if not os.path.isfile(args.video):
        print(f"error: video file not found: {args.video}", file=sys.stderr)
        return 1
    file_size = os.path.getsize(args.video)
    mime_type = mimetypes.guess_type(args.video)[0] or "video/mp4"

    body = _build_video_body(args)
    idempotency_key = _idempotency_key(args.channel_id, args.episode_id, args.video)

    conn = _db()
    try:
        existing = _get_operation(conn, idempotency_key)
        if existing and existing["state"] == "confirmed":
            print(f"already confirmed: video id {existing['external_id']} "
                  f"(idempotency_key={idempotency_key}); not uploading again.")
            return 0

        if not args.confirm_upload:
            _upsert_operation(conn, idempotency_key, args.channel_id, args.episode_id,
                              state="pending", request={"video_body": body, "file_size": file_size,
                                                        "mime_type": mime_type, "video_path": args.video})
            print("DRY RUN (no network call made) -- operations row written as 'pending'.")
            print(f"  idempotency_key: {idempotency_key}")
            print(f"  would upload:    {args.video} ({file_size} bytes, {mime_type})")
            print(f"  video resource:  {json.dumps(body, indent=2)}")
            print("Pass --confirm-upload to actually call the YouTube API. Never pass it this week.")
            return 0

        # ---- real path: never reached without --confirm-upload ----
        client_id, client_secret = _read_oauth_client()
        refresh_token = _read_secret(args.refresh_token_secret, "YOUTUBE_REFRESH_TOKEN_A")
        if not refresh_token:
            raise UploadError(
                f"no refresh token at /run/secrets/{args.refresh_token_secret} or env "
                "YOUTUBE_REFRESH_TOKEN_A -- run scripts/youtube_auth.py's consent flow first"
            )

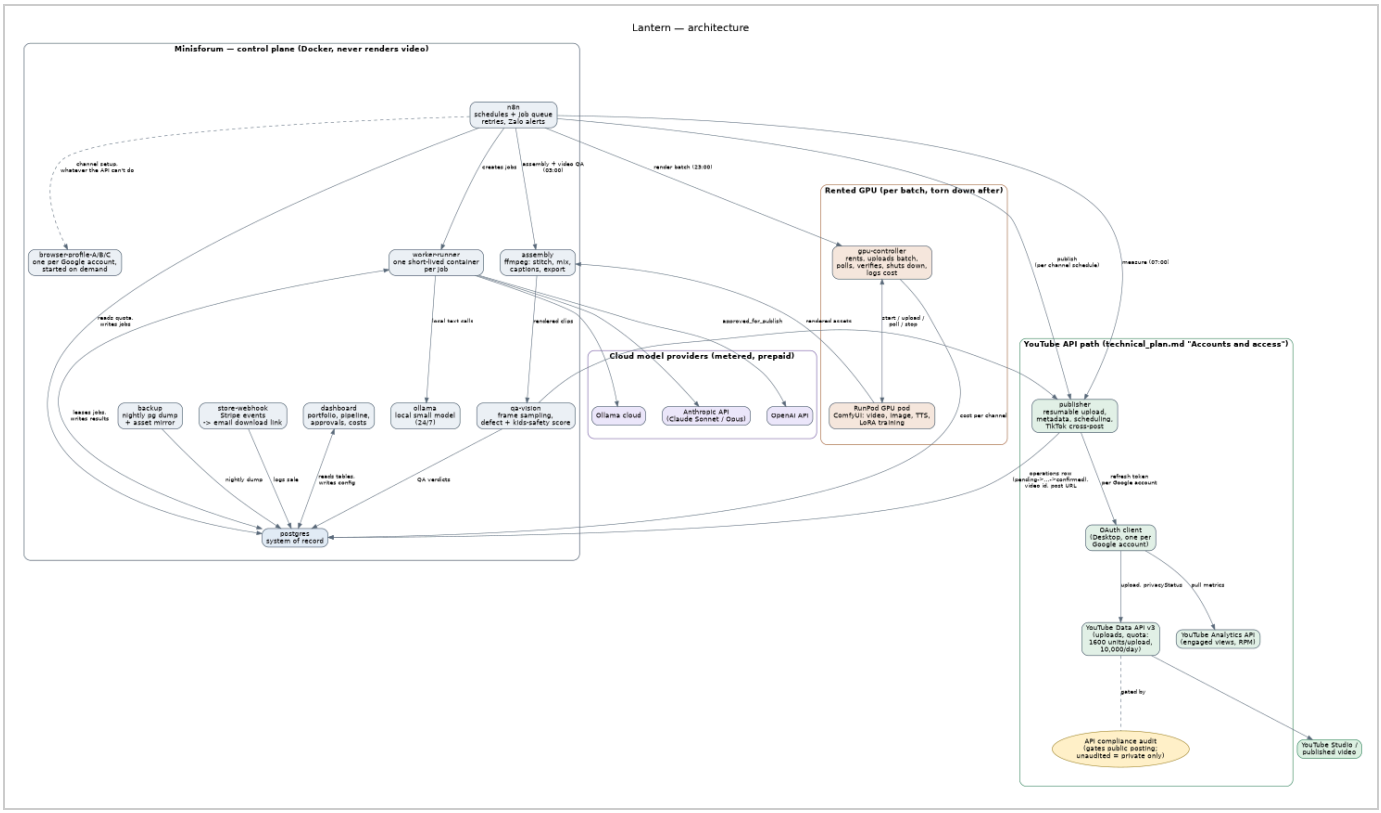
        if existing and existing["state"] in ("in_progress", "unknown") and existing["request"].get("resumable_uri"):
            # Reconcile: resume the existing session instead of starting over.
            session_uri = existing["request"]["resumable_uri"]
            access_token = _refresh_access_token(client_id, client_secret, refresh_token)
            _upsert_operation(conn, idempotency_key, args.channel_id, args.episode_id,
                              state="in_progress", request=existing["request"])
            offset = _query_resumable_offset(session_uri, file_size)
        else:
            access_token = _refresh_access_token(client_id, client_secret, refresh_token)
            session_uri = _create_resumable_session(access_token, body, file_size, mime_type)
            request_record = {"video_body": body, "file_size": file_size, "mime_type": mime_type,
                              "video_path": args.video, "resumable_uri": session_uri}
            _upsert_operation(conn, idempotency_key, args.channel_id, args.episode_id,
                              state="in_progress", request=request_record)
            offset = 0

        try:
            result = _upload_chunks(session_uri, args.video, file_size, mime_type, offset)
        except UploadError as exc:
            # Dropped connection etc: state goes to 'unknown', not 'failed' --
            # per technical_plan.md, an unknown operation is reconciled with
            # the provider (resumed here) before ever being retried as new.
            _upsert_operation(conn, idempotency_key, args.channel_id, args.episode_id,
                              state="unknown",
                              request={"video_body": body, "file_size": file_size, "mime_type": mime_type,
                                       "video_path": args.video, "resumable_uri": session_uri},
                              response={"error": str(exc)})
            print(f"error: {exc} -- operation left 'unknown', re-run to resume.", file=sys.stderr)
            return 2
```

```
        video_id = result.get("id")
        _upsert_operation(conn, idempotency_key, args.channel_id, args.episode_id,
                          state="confirmed" if video_id else "failed",
                          request={"video_body": body, "file_size": file_size, "mime_type": mime_type,
                                   "video_path": args.video, "resumable_uri": session_uri},
                          response=result, external_id=video_id)
        if video_id:
            print(f"uploaded: video id {video_id} (privacyStatus={args.privacy_status})")
            return 0
        print(f"error: upload response had no video id: {result}", file=sys.stderr)
        return 1
    finally:
        conn.close()


if __name__ == "__main__":
    sys.exit(main())
```

(c) Architecture diagram



(d) Content lifecycle / user-flow diagram

